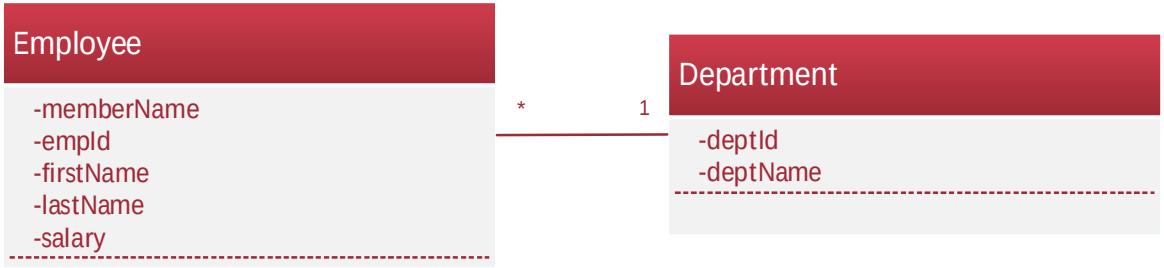
JPA, Part 2

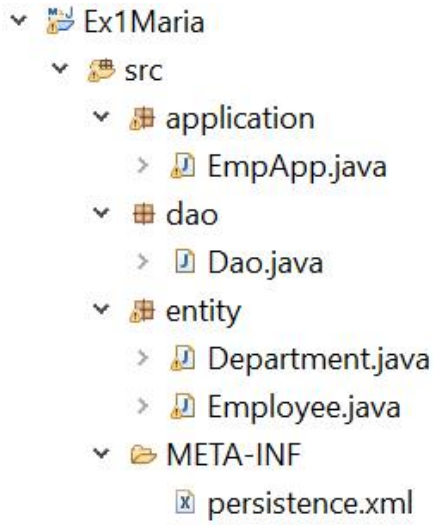
1:M association.

Learning goals

1. Learn to implement 1:M association type with JPA.
2. Learn to know and take advantage of the save and load options provided by JPA.

1:M



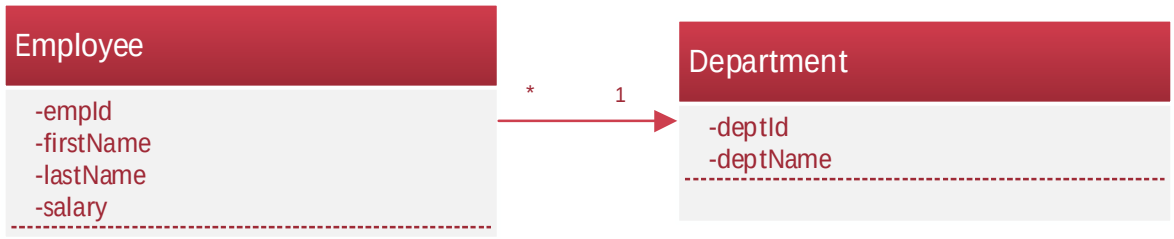


- In a Java application, one-to-many is a common association type.
- An association relationship is implemented by instance variables.
- Examples of one-to-many associations:
  - An employee works at a department. Each department can have many employees.
  - In an adventure game, each item is in a terrain tile. Each terrain tile can accommodate many items.

Navigability

- An association can be navigable either in
  - one direction (uni-directional association), or
  - both directions (bi-directional association).
- Navigability defines, who knows who in the association relationship.
  - 'Knowing' means that an object has an instance variable referring to the other object(s).
- Navigability is indicated by an arrowhead in the class diagram.
- Irrespective of the navigability of the association in the application, a 1:M relationship is implemented by a foreign key in the 'many' end of the relationship at the DB level.
  - The object that takes care of the association is called the owner.
  - A foreign key value is saved or updated whenever the owner object is persisted.
  - In a bi-directional association, it is up to the developer to choose which party is the owner.

Case 1: an employee knows the department (M:1)



- This case has the most straightforward Java implementation.
  - An instance variable is added into the 'many' end.
  - It must be an object reference (of type Department, not int).

@ManyToOne

Employee.java

```
@Entity
public class Employee implements Serializable {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;
    private String firstName;
    private String lastName;
    private double salary;
    @ManyToOne
    private Department department;
```

Department.java

```
@Entity
public class Department {

    @Id
    private int deptId;
    private String deptName;
```

- In the Employee class, add a @ManyToOne annotation.
- As a consequence, associations (implemented by a foreign key) will be updated automatically.

@ManyToOne

```
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: select department_.deptId, department_.deptName as deptName2_0_ from Department department_ where department_.deptId=?
Hibernate: insert into Employee (department_deptId, firstName, lastName, salary) values (?, ?, ?, ?)
Hibernate: select department_.deptId, department_.deptName as deptName2_0_ from Department department_ where department_.deptId=?
Hibernate: insert into Employee (department_deptId, firstName, lastName, salary) values (?, ?, ?, ?)
Hibernate: select department_.deptId, department_.deptName as deptName2_0_ from Department department_ where department_.deptId=?
Hibernate: insert into Employee (department_deptId, firstName, lastName, salary) values (?, ?, ?, ?)
```

- Saving a department generates an INSERT.
- Saving an employee generates a SELECT followed by and INSERT.
  - The SELECT statement verifies the presence of the referenced entity.

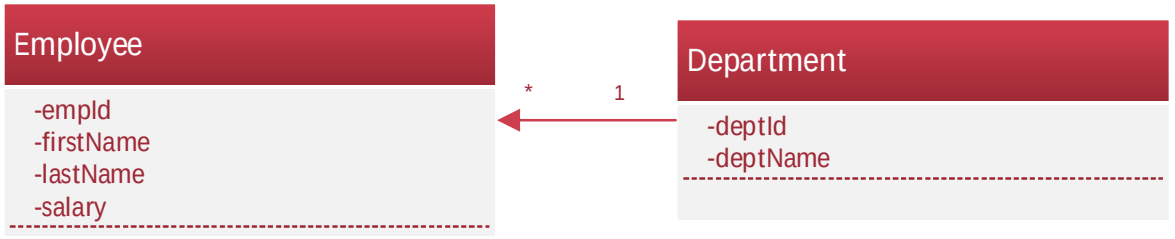
company.department: 2 Next

deptId	deptName
1	Marketing
2	Technology

company.employee: 3 r Next Show all Sorting

empId	firstName	lastName	salary	department_deptId
1	Mary	Smith	3 600	1
2	Urpo	Koski	2 600	2
3	Olga	Glebova	3 110	2

Case 2: a department knows its employees (1:M)



- Now, a data structure class is needed.
  - JPA supports Java List, Set and Map data structures.
    - A list (List) is ordered but a set (Set) is not.
    - Map is designed for (key, value) pairs.

@OneToMany

Employee.java

```
@Entity
public class Employee  {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;
    private String firstName;
    private String lastName;
    private double salary;
```

Department.java

```
@Entity
public class Department {

    @Id
    private int deptId;
    private String deptName;
    @OneToMany
    @JoinColumn
    private List<Employee> employees = new ArrayList<Employee>();
```

- @OneToMany annotation precedes the declaration of the employee list in the Department class.
- @JoinColumn annotation causes the association to be implemented by a foreign key (which is the logically correct way for a 1:M association).
  - Otherwise, JPA would generate an intermediate table.
  - This can be explicitely declared by @JoinTable.

@OneToMany

```
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: update Employee set employees_deptId=? where empId=?
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: update Employee set employees_deptId=? where empId=?
Hibernate: update Employee set employees_deptId=? where empId=?
```

- Saving an employee yields an INSERT statement.
- Subsequently, saving a department generates an INSERT, followed by one UPDATE per employee.

company.department: 2 Next

deptId	deptName
1	Marketing
2	Technology

company.employee: 3 r Next Show all Sorting

empId	firstName	lastName	salary	department_deptId
1	Mary	Smith	3 600	1
2	Urpo	Koski	2 600	2
3	Olga	Glebova	3 110	2

Rules of thumb: @OneToMany or @ManyToOne?

- In the application design phase, one has to select between a uni-directional (@OneToMany or @ManyToOne) or a bi-directional association.
- The choice should be based on the application's needs: choose the most natural way to model the entities' dependence.
- At the database level, the implementation is the same regardless of the choice.
- In a bi-directional association, same information is stored twice in the application's Java data structures.
  - The developer should ensure the synchronization of the data.
  - This means that the information is to be written and updated via methods that modify the contents of both data structures.

Cascading

- In the previous examples, both the employees and the departments had to be persisted by separate operations.
- The developer has to pay attention not to violate foreign key constraints in the database.
- Cascading allows persisting related entities automatically.
- In the example, we could set the department to cascade-save its employees when a department object is saved.
- The cascade types supported by JPA are: PERSIST, DETACH, MERGE, REFRESH ja REMOVE.
  - ALL selects all of them.

Cascading

Department.java

```
@Entity
public class Department {

    @Id
    private int deptId;
    private String deptName;
    @OneToMany(cascade=CascadeType.ALL)
    @JoinColumn
    private List<Employee> employees = new ArrayList<Employee>();
```

Cascading turned on.

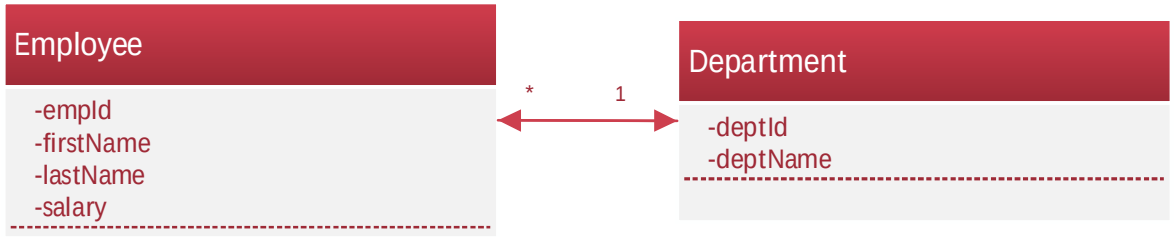
- Note that the employees need not be explicitly saved.

Cascading

```
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: update Employee set employees_deptId=? where empId=?
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: update Employee set employees_deptId=? where empId=?
Hibernate: update Employee set employees_deptId=? where empId=?
```

- Note that Hibernate automatically saves the objects in the correct order.
- Then, it updates the foreign keys.

Case 3: both entities know each other



- Now, both classes need an instance variable referring to the other class.

@OneToMany and @ManyToOne

Employee.java

```
@Entity
public class Employee  {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;
    private String firstName;
    private String lastName;
    private double salary;
    @ManyToOne
    private Department department;
```

Department.java

```
@Entity
public class Department {

    @Id
    private int deptId;
    private String deptName;
    @OneToMany(mappedBy="department")
    private List<Employee> employees = new ArrayList<Employee>();
```

- In the bidirectional case, an owner for the relationship must be selected (either Employee or Department).
- In the example, Employee is the owner.
  - This is indicated by mappedBy in the non-owner class.
  - Generally, choose the 'many' party as the owner.
    - Save operation generates less DB operations.
  - The owner is responsible for saving the association data.
    - As an Employee is saved, the information about the Department is saved as well.

@OneToMany and @ManyToOne

```
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: insert into Department (deptName, deptId) values (?, ?)
Hibernate: select department_.deptId, department_.deptName as deptName2_0_ from Department department_ where department_.deptId=?
Hibernate: insert into Employee (department_deptId, firstName, lastName, salary) values (?, ?, ?, ?)
Hibernate: select department_.deptId, department_.deptName as deptName2_0_ from Department department_ where department_.deptId=?
Hibernate: insert into Employee (department_deptId, firstName, lastName, salary) values (?, ?, ?, ?)
Hibernate: select department_.deptId, department_.deptName as deptName2_0_ from Department department_ where department_.deptId=?
Hibernate: insert into Employee (department_deptId, firstName, lastName, salary) values (?, ?, ?, ?)
```

- Again, the same database structure emerges.
- The association information is saved as part of the INSERT statements that save the employees.

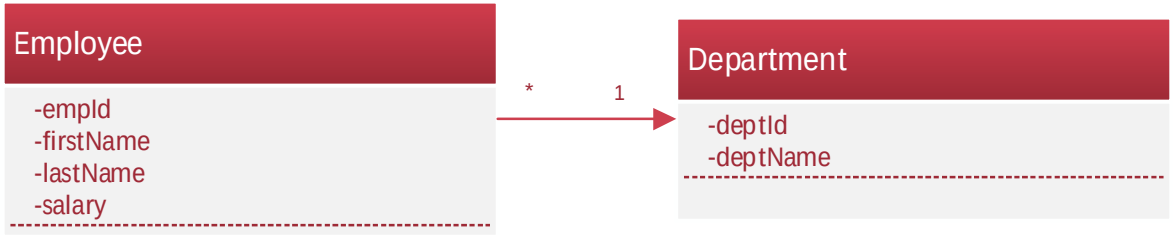
company.department: 2 Next

deptId	deptName
1	Marketing
2	Technology

company.employee: 3 r Next Show all Sorting

empId	firstName	lastName	salary	department_deptId
1	Mary	Smith	3 600	1
2	Urpo	Koski	2 600	2
3	Olga	Glebova	3 110	2

Eager and lazy loading



- Let's consider the situation where an Employee knows its Department (Case 1 before).
- When an Employee is loaded, the correponding Department can be handled in two alternative ways:
  A. Eagerly
     - A Department is loaded immediately at the same time as the Employee.
  B. Lazily
     - A Department is loaded only when it's needed. Until then, it is represented by an unloaded proxy where only the identifying field is set.
       - A proxy is a runtime subclass with modified getters that ask the EntityManager to load the real object.
- This can have an effect on performance.

Eager and lazy loading

- When SQL echoing is on, you can see the effect on the DB queries.
- The fetchtype is defined by an attribute in the annotation (e.g. @ManyToOne)
  - **`fetch=FetchType.LAZY`** or **`fetch=FetchType.EAGER`**
- The default value depends on the type of the association.
  - In a @OneToMany association the default is LAZY.
  - In a @ManyToOne association the default is EAGER.

A pitfall in lazy loading

```
Hibernate: select employee0_.empId as empId1_1_0_, employee0_.department_deptId as departme5_1_0_, employee0_.fi
Urpo
Exception in thread "main" org.hibernate.LazyInitializationException: could not initialize proxy - no Session
        at org.hibernate.proxy.AbstractLazyInitializer.initialize(AbstractLazyInitializer.java:146)
        at org.hibernate.proxy.AbstractLazyInitializer.getImplementation(AbstractLazyInitializer.java:259)
        at org.hibernate.proxy.pojo.javassist.JavassistLazyInitializer.invoke(JavassistLazyInitializer.java:73)
        at entity.Department_$$_jvstebb_1.getDeptName(Department_$$_jvstebb_1.java)
        at application.EmpApp.main(EmpApp.java:37)
```

- When lazy loading is on, the related entity is represented by an unloaded proxy.
- When the data is read via the unloaded proxy, the EntityManager carries out the new database query triggered by a call of the unloaded proxy's getter.
- As such data access is attempted outside the scope of the EntityManager, a LazyInitializationException arises.
  - This is a common error for learning JPA developers.
- In Spring Boot / Spring Web / Spring Data JPA environment, the scope of the EntityManager spans the controller method but not beyond.

Rules of thumb: Eager or lazy loading?

- Lazy loading is generally faster, if we consider merely the loading times of the originally-fetched entities.
- Eager loading can be really slow in associations of `@OneToMany` type, as a slow chain reaction can be triggered.
- Eager loading is desirable, if the automatically fetched related entities are needed anyway by the application.
- Lazy loading can slow down the execution at an unpredicted moment.
- Lazy loading is prone to `LazyInitializationException` if the proxy is accessed outside the `EntityManager`'s scope. Eager loading doesn't pose such a challenge.
- Eager loading can be further modified by so-called fetch groups (not covered on this course).